WRITEUP — Aqurate Junior Data Engineer Challenge

This document covers the three topics requested in the challenge brief: data quality issues found and resolved, production monitoring strategy, and AI tool usage. For the full step-by-step engineering log — including every decision, bug fix, and investigation — see PROJECT_JOURNAL.md in the repository.

1. Data Issues in orders_raw

The source dataset contained 9,268 rows across 14 columns. Before writing any pipeline code, the full dataset was explored to catalogue any inconsistency. 9 distinct data quality issues were identified, plus 1 anomaly discovered post-implementation. Each issue was handled with a deliberate strategy — exclude, normalize, infer, or keep-and-flag — depending on whether the data was recoverable.

#	Issue	Rows Affected	Resolution
1	Mixed timestamp formats (ISO 8601, Unix epoch, DD/MM/YYYY HH:MM)	9,268 (all rows)	Normalised all to TIMESTAMPTZ via a cascade parser that tries each format in order
2	True duplicate rows — identical (order_id, sku) with identical values	177 duplicates	Deduplicated — kept first occurrence only
3	Multi-item orders sharing the same order_id (different SKUs)	2,491 order IDs	Preserved — these are valid multi-line orders, not duplicates
4	Negative quantities on completed orders	167 rows	Excluded — a completed sale with qty = −1 is contradictory (likely a mislabelled return)
5	Test orders (status='test', @aqurate.ai emails)	101 rows	Excluded — internal testing data, confirmed by the aqurate.ai email domain
6	NULL customer_id	103 rows	Kept, flagged (had_null_customer = TRUE) — likely guest checkouts; included in revenue, excluded from customer_spend_eur
7	NULL category	79 rows	Inferred from SKU prefix (deterministic mapping: EL→Electronics, BK→Books, etc.), flagged

			with had_null_category = TRUE
8	Zero unit_price on completed orders	24 rows	Kept, flagged (had_zero_price = TRUE) — possible promos; zero × qty = €0, so revenue totals are unaffected
9	Malformed SKUs (e.g. SKUEL001 vs SKU-EL-001)	~few	Normalised via regex to canonical SKU-XX-NNN format
10	Anomalous unit_price = 999,999 on a single order line (ORD12890)	1 row	Kept — removing it would require an arbitrary price cap; flagged in the journal as a production monitoring concern

Cleaning funnel

Stage	Rows	Removed
Raw (orders_raw)	9,268	—
− Test orders	9,167	−101
− Negative quantities	9,000	−167
− True duplicates	**8,823**	**−177**

The final orders_clean table contains 8,823 rows — a 4.8% reduction from the raw dataset. All exclusions are deliberate and documented. Rows that were kept despite quality issues (NULL customer, NULL category, zero price) are explicitly flagged with boolean columns, so downstream consumers can filter them as needed.

2. Production Monitoring Strategy

The question asks: if the daily pipeline silently failed, how would I find out? The answer has three layers — what is already implemented, what I would add as practical next steps, and what a larger enterprise deployment would look like.

What is already in place

- GitHub Actions failure notifications. When any pipeline step throws an exception, the orchestrator (src/pipeline.py) calls sys.exit(1), which marks the GitHub Actions run as failed. GitHub automatically sends an email notification to the repository owner on workflow failure. This catches hard crashes — Python errors, database connection failures, API timeouts. (Many engineers have issues reading their emails, so in my opinion an email notification is not enough 😊)
- pipeline runs an audit table. Every step logs its execution to a pipeline runs table with step name, status (success/failure), row count, duration, and error message. This provides a

quarriable history of every pipeline execution, making it easy to spot patterns (e.g., a step that succeeded but processed 0 rows).

- Fail-fast behavior. If Step 1 fails, Steps 2–5 never run. This prevents downstream tables from being computed on stale or incomplete data.

What I would add next

The existing setup catches hard crashes, but not silent failures — situations where the pipeline runs successfully, but the data it produces is wrong, stale, or incomplete. Here are four practical additions:

Freshness checks (SLA monitoring). A separate lightweight job, scheduled to run one hour after the main pipeline, would query the pipeline_runs table: SELECT MAX(started_at) FROM pipeline_runs WHERE step_name = 'customer_spend_eur' AND status ='success'. If the latest successful run is older than 26 hours, it means today's pipeline either didn't run or failed silently. This catches the scenario where GitHub Actions itself has an outage, and the cron job never fires — something that email alerts would not catch.

Row count assertions (volume checks). Before truncating a target table, the ingest step would validate the fetched row count against a historical baseline. For example: if the API returns fewer than 8,000 rows (our baseline is ~9,200), the pipeline will abort with an explicit error rather than silently loading partial data. This prevents a common failure mode where APIs glitch and return incomplete results, which would wipe out our good data via TRUNCATE + INSERT.

Data quality thresholds. The cleaning step currently excludes test orders, negative quantities, and duplicates. If the exclusion rate suddenly spikes — say from the normal ~5% to 30% — it signals a source data problem. A simple check at the end of the clean step (if drop_off_rate > 0.10: alert) would catch this before the corrupted data flows into analytical tables.

Slack/Teams webhook alerts. In a team setting, email notifications are too slow. A webhook step in the GitHub Actions workflow, triggered on failure, would post a message to a #data-alerts channel. This ensures the team is aware within minutes, not hours.

At enterprise scale

For a larger deployment with dozens of pipelines and multiple data sources, the monitoring would shift to dedicated orchestration and observability tools. Pipeline orchestration tools like Apache Airflow or Prefect provide built-in DAG-level monitoring, automatic retries, and dependency management. Dashboarding tools like Grafana can visualize row counts, execution durations, and error rates over time, with configurable alerting thresholds. Anomaly detection on key metrics — such as the €999,999 unit price discovered during this project — would be handled by automated statistical checks rather than manual investigation.

3. AI Usage

Tools used

- Claude Opus 4.6 — used for deeper, structural tasks: designing the project architecture, writing complex SQL queries (CTEs for multi-country customer handling), implementing the full cleaning pipeline, and generating the GitHub Actions workflow.
- Claude Sonnet 4.6 — used for smaller, faster tasks: quick questions, code explanations, understanding specific principles (e.g., FX rate carry-forward conventions, UPSERT semantics), and minor edits.

How AI was used

AI assisted with code writing and helped me understand some of the underlying principles — for example, how financial markets handle weekend rates. The AI was used as a coding assistant, not as an autonomous agent.

The entire decision-making process was mine. Before accepting any AI-generated code or suggestion, I made sure I understood the task, processed the options, and chose the approach that made sense for the problem. For example, when handling multi-country customers, the AI proposed 2 options (group by country, pick primary silently). My proposal was: pick primary + flag. I chose the flag approach because it preserves information that would otherwise be silently lost.

Active verification

Throughout the process, I actively verified every implementation step. This included reviewing all generated code before committing, running the pipeline locally to confirm outputs matched expectations, investigating unexpected results (such as the €999,999 anomaly, which was discovered through my own review of the customer_spend_eur output — not flagged by AI), and correcting mistakes where they occurred. Before writing this WRITEUP, I conducted a full audit of every file in the repository to ensure consistency and correctness.

Further Information

For the complete record of every engineering decision, data investigation, bug fix, and design trade-off made during this project, see PROJECT_JOURNAL.md in the repository root. The journal contains detailed explanations with real data examples for each of the 10 data quality issues, the FX rate gap-filling logic, the multi-country customer resolution, and the GitHub Actions automation setup.